

Diagnose Root Shell State

ChatGPT - September 8, 2026 at 06:03 PM

Ahhh yes, brother — **I misunderstood the exact failure.** You mean:

- You select text.
- You scroll **short or long** while that selection is highlighted.
- You use the terminal's **auto-copy/selection behavior**.
- The highlight eventually **disappears by itself**.
- You are **not** talking about the ticker resuming after the selection ends.

So the real problem is that **our plugin is still doing something that causes VTE to lose/redraw the selection**, even though the clock is supposed to be paused.

The sticky `selection_pause` approach didn't solve it because we're still polling VTE every 50 ms and potentially touching its state during the selection lifecycle.

The correct direction

We should make the plugin **completely hands-off once a mouse selection starts**:

1. Mouse press → immediately mark that terminal frozen.
2. While frozen:
 - **no `get_has_selection()` polling**
 - **no scroll polling**
 - **no state transitions**
 - **no writes except the initial `paused`**
3. Mouse release → **do not resume yet**.
4. Keep the terminal frozen until we can safely determine that the user has actually cleared the selection / returned to normal.
5. Only then write `live` .

That removes the plugin's 50-ms polling loop from the selection itself, which is the part I should have changed instead of trying to make the existing polling "sticky."

And importantly: **we keep the PTY/session detection exactly as it is**, since that part is now working for both `pts/8` and `pts/12` .

If you want, I'll replace the plugin again with that **selection-lock architecture** and **give you the entire file**, not a snippet.

YOU

yeah, provide full right file so we call it a day

Yes brother. **This version changes only the selection/scroll handling** while keeping the PTY/session detection that was already working for `pts/8` and `pts/12`.

Replace the entire file with this:

Python

```
import os
import time

from gi.repository import GLib
import terminatorlib.plugin as plugin

AVAILABLE = ['LiveClockScroll']

class LiveClockScroll(plugin.Plugin):
    capabilities = []

    def __init__(self):
        self.poll_id = GLib.timeout_add(50, self._poll)

        # PTY -> paused state
        self.paused = {}

        # VTE -> connected handlers
        self.connected = set()

        # VTE -> mouse button currently held
        self.mouse_pressed = set()

        # VTE -> selection is being/was actively manipulated
        self.selection_lock = set()

        # VTE -> time at which mouse was released
        self.release_time = {}

        # Keep selection protection alive briefly after release.
        # This prevents a redraw race immediately after copying.
        self.RELEASE_GRACE = 1.0
```

```
def unload(self):
    if self.poll_id:
        Glib.source_remove(self.poll_id)
        self.poll_id = None

def _read_file(self, path):
    try:
        with open(path, "r") as f:
            return f.read().strip()
    except Exception:
        return None

def _stat_info(self, pid):
    data = self._read_file(
        f"/proc/{pid}/stat"
    )

    if not data:
        return None

    try:
        close = data.rfind("(")

        if close == -1:
            return None

        rest = data[close + 2:].split()

        return {
            "pid": int(pid),
            "ppid": int(rest[1]),
            "pgid": int(rest[2]),
            "sid": int(rest[3]),
            "tty_nr": int(rest[4]),
            "tpgid": int(rest[5]),
        }

    except (ValueError, IndexError):
        return None

def _children(self, pid):
    data = self._read_file(
        f"/proc/{pid}/task/{pid}/children"
    )
```

```
    if not data:
        return []

    result = []

    for item in data.split():
        try:
            result.append(int(item))
        except ValueError:
            pass

    return result

def _tty_nr_to_pts(self, tty_nr):

    try:
        tty_nr = int(tty_nr)
    except (TypeError, ValueError):
        return None

    if tty_nr == 0:
        return None

    major = (tty_nr >> 8) & 0xff
    minor = tty_nr & 0xff

    if 136 <= major <= 143:
        return str(
            (major - 136) * 256 + minor
        )

    return None

def _collect_descendants(self, root_pid):

    result = []

    queue = [root_pid]
    visited = set()

    while queue:

        pid = queue.pop(0)
```

```
        if pid in visited:
            continue

        visited.add(pid)

        info = self._stat_info(pid)

        if info is not None:
            result.append(info)

        for child in self._children(pid):

            if child not in visited:
                queue.append(child)

    return result

def _terminal_pts(self, terminal):

    try:
        root_pid = getattr(
            terminal,
            "pid",
            None
        )

        if not root_pid:
            return None

        root_pid = int(root_pid)

    except Exception:
        return None

    processes = self._collect_descendants(
        root_pid
    )

    if not processes:
        return None

    sessions = {}
```

```
for info in processes:

    sid = info["sid"]

    if sid not in sessions:

        sessions[sid] = {
            "sid": sid,
            "depth": -1,
            "pts": None,
            "members": [],
        }

        sessions[sid]["members"].append(
            info
        )

by_pid = {
    info["pid"]: info
    for info in processes
}

depth = {
    root_pid: 0
}

changed = True

while changed:

    changed = False

    for info in processes:

        pid = info["pid"]

        if pid in depth:
            continue

        parent = info["ppid"]

        if parent in depth:

            depth[pid] = (
```



```
        depth[parent] + 1
    )

    changed = True

for sid, session in sessions.items():

    best_depth = -1
    best_pts = None

    for info in session["members"]:

        pid = info["pid"]

        d = depth.get(
            pid,
            0
        )

        pts = self._tty_nr_to_pts(
            info["tty_nr"]
        )

        if (
            pts is not None
            and d > best_depth
        ):

            best_depth = d
            best_pts = pts

    session["depth"] = best_depth
    session["pts"] = best_pts

root_info = by_pid.get(
    root_pid
)

if root_info is None:
    return None

root_sid = root_info["sid"]

candidates = [
```

```
        session
        for session in sessions.values()
        if session["pts"] is not None
    ]

    if not candidates:
        return None

    nested = [
        session
        for session in candidates
        if session["sid"] != root_sid
    ]

    if nested:

        nested.sort(
            key=lambda s: (
                s["depth"],
                s["sid"]
            ),
            reverse=True
        )

        return nested[0]["pts"]

    root_session = sessions.get(
        root_sid
    )

    if root_session:
        return root_session["pts"]

    return candidates[0]["pts"]

def _debug(self, message):

    try:

        with open(
            "/tmp/live-clock-debug",
            "a"
        ) as f:
```

```
        f.write(
            message + "\n"
        )

    except Exception:
        pass

def _write_state(self, pts, state):

    if not pts:
        return

    state_file = (
        f"/tmp/zsh-live-clock-{pts}"
    )

    tmp_file = (
        state_file + ".tmp"
    )

    try:

        with open(
            tmp_file,
            "w"
        ) as f:

            f.write(state)

            os.replace(
                tmp_file,
                state_file
            )

    except OSError:
        pass

def _pause_terminal(self, terminal):

    pts = self._terminal_pts(
        terminal
    )

    if not pts:
```

```
        return

    self.paused[pts] = True

    self._write_state(
        pts,
        "paused"
    )

    self._debug(
        f"PAUSE: pts={pts}"
    )

def _mouse_press(
    self,
    vte,
    event,
    terminal
):

    vte_id = id(vte)

    self.mouse_pressed.add(
        vte_id
    )

    # A mouse press means the terminal is entering
    # an interaction. Freeze immediately.
    self.selection_lock.add(
        vte_id
    )

    self.release_time.pop(
        vte_id,
        None
    )

    self._pause_terminal(
        terminal
    )

    self._debug(
        "MOUSE PRESS -> "
        f"LOCK vte={vte_id}"
    )
```

```
)

return False

def _mouse_release(
    self,
    vte,
    event,
    terminal
):

    vte_id = id(vte)

    self.mouse_pressed.discard(
        vte_id
    )

    # IMPORTANT:
    # Do NOT resume here.
    #
    # The selection/copy operation can still be completing
    # immediately after button release. Resuming here can
    # cause reset-prompt/redraw and make VTE lose its highlight.
    self.release_time[vte_id] = time.monotonic()

    self._debug(
        "MOUSE RELEASE -> "
        f"LOCK RETAINED vte={vte_id}"
    )

    return False

def _connect_mouse_handlers(
    self,
    terminal,
    vte
):

    key = id(vte)

    if key in self.connected:
        return

    try:
```

```
        vte.connect(
            "button-press-event",
            self._mouse_press,
            terminal
        )

        vte.connect(
            "button-release-event",
            self._mouse_release,
            terminal
        )

        self.connected.add(
            key
        )

        self._debug(
            f"MOUSE CONNECTED: vte={key}"
        )

    except Exception as e:

        self._debug(
            "MOUSE CONNECT ERROR: "
            f"{type(e).__name__}: {e}"
        )

    def _get_vte_state(self, vte):

        try:

            adjustment = (
                vte.get_vadjustment()
            )

            upper = (
                adjustment.get_upper()
            )

            page = (
                adjustment.get_page_size()
            )
```

```
        value = (
            adjustment.get_value()
        )

        bottom = max(
            0,
            upper - page
        )

        scrolled = (
            value < bottom - 0.5
        )

    except Exception as e:

        self._debug(
            "SCROLL ERROR: "
            f"{type(e).__name__}: {e}"
        )

        scrolled = False

    try:

        selected = (
            vte.get_has_selection()
        )

    except Exception:

        selected = False

    return (
        scrolled,
        selected
    )

def _resume(self, pts):

    if not pts:
        return

    if self.paused.get(pts) is not False:
```

```
self.paused[pts] = False

self._write_state(
    pts,
    "live"
)

self._debug(
    f"RESUME: pts={pts}"
)

def _poll(self):

    try:

        import terminatorlib.terminator as terminatorlib

        terminator = (
            terminatorlib.Terminator()
        )

        now = time.monotonic()

        for terminal in terminator.terminals:

            vte = terminal.get_vte()

            vte_id = id(vte)

            pts = self._terminal_pts(
                terminal
            )

            self._debug(
                "VTE DEBUG: "
                f"terminal.id={id(terminal)} "
                f"terminal.pid="
                f"{getattr(terminal, 'pid', None)} "
                f"active_pts={pts} "
                f"vte_type={type(vte).__name__}"
            )

            self._connect_mouse_handlers(
                terminal,
```



```
        vte
    )

    if not pts:
        continue

    # -----
    # HARD SELECTION/INTERACTION LOCK
    # -----
    #
    # Once a mouse interaction has happened, do NOT
    # continuously interrogate VTE selection state.
    #
    # This is the important change. Polling
    # get_has_selection() during VTE's selection/copy/
    # scroll processing can race against VTE's own
    # selection handling.
    # -----

    if vte_id in self.mouse_pressed:

        if self.paused.get(pts) is not True:

            self.paused[pts] = True

            self._write_state(
                pts,
                "paused"
            )

            self._debug(
                f"PAUSE: mouse held pts={pts}"
            )

            continue

    if vte_id in self.selection_lock:

        released_at = (
            self.release_time.get(
                vte_id
            )
        )
```

```
# Still inside the post-release protection
# window. Absolutely no VTE state manipulation.
if (
    released_at is None
    or
    now - released_at
    < self.RELEASE_GRACE
):

    if (
        self.paused.get(pts)
        is not True
    ):

        self.paused[pts] = True

        self._write_state(
            pts,
            "paused"
        )

        continue

# After the grace period, inspect VTE once.
#
# If text is still selected OR the terminal is
# still scrolled, remain locked indefinitely.
try:

    scrolled, selected = (
        self._get_vte_state(
            vte
        )
    )

except Exception:

    scrolled = True
    selected = True

self._debug(
    "LOCK CHECK: "
    f"pts={pts} "
    f"scrolled={scrolled} "
```

```
f"selected={selected}"
)

if scrolled or selected:

    # Keep the lock.
    if (
        self.paused.get(pts)
        is not True
    ):

        self.paused[pts] = True

        self._write_state(
            pts,
            "paused"
        )

        continue

    # No selection and back at the bottom.
    #
    # This is the safe point to release the lock.
    self.selection_lock.discard(
        vte_id
    )

    self.release_time.pop(
        vte_id,
        None
    )

    self._debug(
        "LOCK CLEAR: "
        f"vte={vte_id}"
    )

    self._resume(
        pts
    )

    continue

# -----
```

```
# NORMAL SCROLL DETECTION
# -----

    scrolled, selected = (
        self._get_vte_state(
            vte
        )
    )

    if scrolled or selected:

        if (
            self.paused.get(pts)
            is not True
        ):

            self.paused[pts] = True

            self._write_state(
                pts,
                "paused"
            )

            self._debug(
                "PAUSE: "
                f"pts={pts} "
                f"scrolled={scrolled} "
                f"selected={selected}"
            )

            continue

# -----
# NORMAL LIVE STATE
# -----

    self._resume(
        pts
    )

except Exception as e:

    self._debug(
        "POLL ERROR: "
```

```

        f"{type(e).__name__}: {e}"
    )

    return True

```

Then:

Bash

```
python3 -m py_compile ~/.config/terminator/plugins/live_clock_scroll.py
```

If that gives no output:

Bash

```

pkill terminator
terminator &

```

The important difference

This time, once you **mouse-select and release**, the plugin does **not immediately resume the clock**, does **not call `reset-prompt`**, and does **not repeatedly inspect the selection during the critical copy/scroll interaction**.

It gives VTE a **1-second protection window**, then checks:

- still selected → stays paused
- still scrolled → stays paused
- selection cleared + back at bottom → resumes

So this specifically targets the **highlight disappearing after short/long scroll + auto-copy** problem.

That should be the final test, brother.

you

auto unhighlight selected area still not happening automatically upon left key release man, fix it ,

Ahhh **now I have it exactly, brother**. You want the selected highlight to **disappear automatically when the left mouse button is released**. The text should already have been auto-copied, but the visual selection must be cleared.

So we don't need more state detection for this part. On **button-release-event** , we explicitly tell VTE to **unselect**.

Replace the **whole plugin** with this:

Python

```
import os
import time

from gi.repository import GLib
import terminatorlib.plugin as plugin

AVAILABLE = ['LiveClockScroll']

class LiveClockScroll(plugin.Plugin):
    capabilities = []

    def __init__(self):
        self.poll_id = GLib.timeout_add(50, self._poll)

        # PTY -> paused state
        self.paused = {}

        # VTE -> connected handlers
        self.connected = set()

        # VTE -> mouse button currently held
        self.mouse_pressed = set()

        # VTE -> interaction lock
        self.selection_lock = set()

        # VTE -> release timestamp
        self.release_time = {}
```

```
# Small protection period so the ticker cannot redraw
# while VTE finishes its selection/copy operation.
self.RELEASE_GRACE = 0.25

def unload(self):
    if self.poll_id:
        GLib.source_remove(self.poll_id)
        self.poll_id = None

def _read_file(self, path):
    try:
        with open(path, "r") as f:
            return f.read().strip()
    except Exception:
        return None

def _stat_info(self, pid):
    data = self._read_file(
        f"/proc/{pid}/stat"
    )

    if not data:
        return None

    try:
        close = data.rfind("(")

        if close == -1:
            return None

        rest = data[close + 2:].split()

        return {
            "pid": int(pid),
            "ppid": int(rest[1]),
            "pgid": int(rest[2]),
            "sid": int(rest[3]),
            "tty_nr": int(rest[4]),
            "tpgid": int(rest[5]),
        }

    except (ValueError, IndexError):
        return None
```

```
def _children(self, pid):
    data = self._read_file(
        f"/proc/{pid}/task/{pid}/children"
    )

    if not data:
        return []

    result = []

    for item in data.split():
        try:
            result.append(int(item))
        except ValueError:
            pass

    return result

def _tty_nr_to_pts(self, tty_nr):

    try:
        tty_nr = int(tty_nr)
    except (TypeError, ValueError):
        return None

    if tty_nr == 0:
        return None

    major = (tty_nr >> 8) & 0xff
    minor = tty_nr & 0xff

    if 136 <= major <= 143:
        return str(
            (major - 136) * 256 + minor
        )

    return None

def _collect_descendants(self, root_pid):

    result = []

    queue = [root_pid]
    visited = set()
```



```
while queue:

    pid = queue.pop(0)

    if pid in visited:
        continue

    visited.add(pid)

    info = self._stat_info(pid)

    if info is not None:
        result.append(info)

    for child in self._children(pid):

        if child not in visited:
            queue.append(child)

    return result

def _terminal_pts(self, terminal):

    try:
        root_pid = getattr(
            terminal,
            "pid",
            None
        )

        if not root_pid:
            return None

        root_pid = int(root_pid)

    except Exception:
        return None

    processes = self._collect_descendants(
        root_pid
    )

    if not processes:
```

```
        return None

sessions = {}

for info in processes:

    sid = info["sid"]

    if sid not in sessions:

        sessions[sid] = {
            "sid": sid,
            "depth": -1,
            "pts": None,
            "members": [],
        }

        sessions[sid]["members"].append(
            info
        )

by_pid = {
    info["pid"]: info
    for info in processes
}

depth = {
    root_pid: 0
}

changed = True

while changed:

    changed = False

    for info in processes:

        pid = info["pid"]

        if pid in depth:
            continue

        parent = info["ppid"]
```

```
        if parent in depth:

            depth[pid] = (
                depth[parent] + 1
            )

            changed = True

    for sid, session in sessions.items():

        best_depth = -1
        best_pts = None

        for info in session["members"]:

            pid = info["pid"]

            d = depth.get(
                pid,
                0
            )

            pts = self._tty_nr_to_pts(
                info["tty_nr"]
            )

            if (
                pts is not None
                and d > best_depth
            ):

                best_depth = d
                best_pts = pts

        session["depth"] = best_depth
        session["pts"] = best_pts

    root_info = by_pid.get(
        root_pid
    )

    if root_info is None:
        return None
```

```
root_sid = root_info["sid"]

candidates = [
    session
    for session in sessions.values()
    if session["pts"] is not None
]

if not candidates:
    return None

nested = [
    session
    for session in candidates
    if session["sid"] != root_sid
]

if nested:

    nested.sort(
        key=lambda s: (
            s["depth"],
            s["sid"]
        ),
        reverse=True
    )

    return nested[0]["pts"]

root_session = sessions.get(
    root_sid
)

if root_session:
    return root_session["pts"]

return candidates[0]["pts"]

def _debug(self, message):

    try:

        with open(
```

```
        "/tmp/live-clock-debug",
        "a"
    ) as f:

        f.write(
            message + "\n"
        )

    except Exception:
        pass

def _write_state(self, pts, state):

    if not pts:
        return

    state_file = (
        f"/tmp/zsh-live-clock-{pts}"
    )

    tmp_file = (
        state_file + ".tmp"
    )

    try:

        with open(
            tmp_file,
            "w"
        ) as f:

            f.write(state)

            os.replace(
                tmp_file,
                state_file
            )

    except OSError:
        pass

def _pause_terminal(self, terminal):

    pts = self._terminal_pts(
```

```
        terminal
    )

    if not pts:
        return

    self.paused[pts] = True

    self._write_state(
        pts,
        "paused"
    )

    self._debug(
        f"PAUSE: pts={pts}"
    )

def _resume(self, pts):

    if not pts:
        return

    if self.paused.get(pts) is not False:

        self.paused[pts] = False

        self._write_state(
            pts,
            "live"
        )

        self._debug(
            f"RESUME: pts={pts}"
        )

def _clear_selection(self, vte):

    try:
        # VTE's selection is cleared explicitly here.
        #
        # Clipboard contents are unaffected: the selected
        # text has already been copied by the terminal's
        # normal selection/autocopy mechanism.
        vte.unselect_all()
```

```
        self._debug(
            f"UNSELECT: vte={id(vte)}"
        )

    except Exception as e:

        self._debug(
            "UNSELECT ERROR: "
            f"{type(e).__name__}: {e}"
        )

def _mouse_press(
    self,
    vte,
    event,
    terminal
):

    vte_id = id(vte)

    self.mouse_pressed.add(
        vte_id
    )

    self.selection_lock.add(
        vte_id
    )

    self.release_time.pop(
        vte_id,
        None
    )

    self._pause_terminal(
        terminal
    )

    self._debug(
        "MOUSE PRESS -> "
        f"LOCK vte={vte_id}"
    )

    return False
```

```
def _mouse_release(
    self,
    vte,
    event,
    terminal
):

    vte_id = id(vte)

    self.mouse_pressed.discard(
        vte_id
    )

    self.release_time[vte_id] = (
        time.monotonic()
    )

    self._debug(
        "MOUSE RELEASE -> "
        f"UNSELECT vte={vte_id}"
    )

    # Let VTE finish its normal mouse-release and
    # autocopy processing first.
    #
    # Then explicitly remove the visual selection.
    GLib.idle_add(
        self._clear_selection,
        vte
    )

    return False

def _connect_mouse_handlers(
    self,
    terminal,
    vte
):

    key = id(vte)

    if key in self.connected:
        return
```



```
try:

    vte.connect(
        "button-press-event",
        self._mouse_press,
        terminal
    )

    vte.connect(
        "button-release-event",
        self._mouse_release,
        terminal
    )

    self.connected.add(
        key
    )

    self._debug(
        f"MOUSE CONNECTED: vte={key}"
    )

except Exception as e:

    self._debug(
        "MOUSE CONNECT ERROR: "
        f"{type(e).__name__}: {e}"
    )

def _get_vte_state(self, vte):

    try:

        adjustment = (
            vte.get_vadjustment()
        )

        upper = (
            adjustment.get_upper()
        )

        page = (
            adjustment.get_page_size()
```

```
)

value = (
    adjustment.get_value()
)

bottom = max(
    0,
    upper - page
)

scrolled = (
    value < bottom - 0.5
)

except Exception as e:

    self._debug(
        "SCROLL ERROR: "
        f"{type(e).__name__}: {e}"
    )

    scrolled = False

try:

    selected = (
        vte.get_has_selection()
    )

except Exception:

    selected = False

return (
    scrolled,
    selected
)

def _poll(self):

    try:

        import terminatorlib.terminator as terminatorlib
```

```
terminator = (  
    terminatorlib.Terminator()  
)  
  
now = time.monotonic()  
  
for terminal in terminator.terminals:  
  
    vte = terminal.get_vte()  
  
    vte_id = id(vte)  
  
    pts = self._terminal_pts(  
        terminal  
    )  
  
    self._debug(  
        "VTE DEBUG: "  
        f"terminal.id={id(terminal)} "  
        f"terminal.pid="  
        f"{getattr(terminal, 'pid', None)} "  
        f"active_pts={pts} "  
        f"vte_type={type(vte).__name__}"  
    )  
  
    self._connect_mouse_handlers(  
        terminal,  
        vte  
    )  
  
    if not pts:  
        continue  
  
    # -----  
    # MOUSE BUTTON HELD  
    # -----  
  
    if vte_id in self.mouse_pressed:  
  
        if (  
            self.paused.get(pts)  
            is not True  
        ):  

```

```
        self.paused[pts] = True

        self._write_state(
            pts,
            "paused"
        )

        continue

# -----
# POST-RELEASE PROTECTION
# -----

if vte_id in self.selection_lock:

    released_at = (
        self.release_time.get(
            vte_id
        )
    )

    if (
        released_at is None
        or
        now - released_at
        < self.RELEASE_GRACE
    ):

        if (
            self.paused.get(pts)
            is not True
        ):

            self.paused[pts] = True

            self._write_state(
                pts,
                "paused"
            )

            continue

# The visual selection has already been
```

```
# explicitly cleared by _clear_selection().
#
# Now check only whether the terminal is
# still scrolled.
scrolled, selected = (
    self._get_vte_state(
        vte
    )
)

if scrolled:

    # Keep ticker paused while user remains
    # away from the bottom.
    if (
        self.paused.get(pts)
        is not True
    ):

        self.paused[pts] = True

        self._write_state(
            pts,
            "paused"
        )

        continue

# Selection is no longer relevant because
# we explicitly cleared it on release.
self.selection_lock.discard(
    vte_id
)

self.release_time.pop(
    vte_id,
    None
)

self._debug(
    "LOCK CLEAR: "
    f"vte={vte_id}"
)
```

```
        self._resume(
            pts
        )

        continue

# -----
# NORMAL SCROLL / SELECTION DETECTION
# -----

        scrolled, selected = (
            self._get_vte_state(
                vte
            )
        )

        if scrolled or selected:

            if (
                self.paused.get(pts)
                is not True
            ):

                self.paused[pts] = True

                self._write_state(
                    pts,
                    "paused"
                )

                self._debug(
                    "PAUSE: "
                    f"pts={pts} "
                    f"scrolled={scrolled} "
                    f"selected={selected}"
                )

            continue

# -----
# NORMAL LIVE STATE
# -----

        self._resume(
```

```
        pts
    )

    except Exception as e:

        self._debug(
            "POLL ERROR: "
            f"{type(e).__name__}: {e}"
        )

    return True
```

Then:

Bash

```
python3 -m py_compile ~/.config/terminator/plugins/live_clock_scroll.py
```

If clean:

Bash

```
pkill terminator
terminator &
```

Now the behavior is exactly:

Left mouse down → select → left mouse up → auto-copy happens → `unselect_all()` clears the highlight.

The clipboard contents aren't intentionally cleared; we're only removing the **visual VTE selection**.

And the clock remains protected during that mouse interaction so the ticker doesn't interfere with the copy operation.